Bézier Curve — Matrix Method (Symbolic)

$P(u) = U \cdot M \cdot G$ via Symbolics.jl

Code Explanation for Julia Beginners

`05-bezier-matrix-sympy-text.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the Julia code in `05-bezier-matrix-sympy-text.jl` for readers new to Julia. Script 05 is a deliberate hybrid: it takes the straight output structure of script 04 and substitutes exact `Rational{Int}` arithmetic throughout, using Symbolics.jl only for polynomial formatting. This document focuses on what changed relative to scripts 02 and 04, which are its two parents. Shared components are listed by reference.

Contents

1 Type Map: How the Pieces Fit Together **2**

2 What Is Shared (by Reference) **3**

3 The `BezierMatrix` Struct — Type Change **3**

4 `derive_M` and `evaluate_all` — Type Change Only **4**
- 4.1 `derive_M` 4
- 4.2 `evaluate_all` 4

5 `fmt_rat` — The Rational Formatter **5**

6 `fmt_poly` — Building a Symbolic Expression **5**
- 6.1 What it does 5

6.2 The generator sum with `if` filter 5
6.3 The `init` keyword argument 6

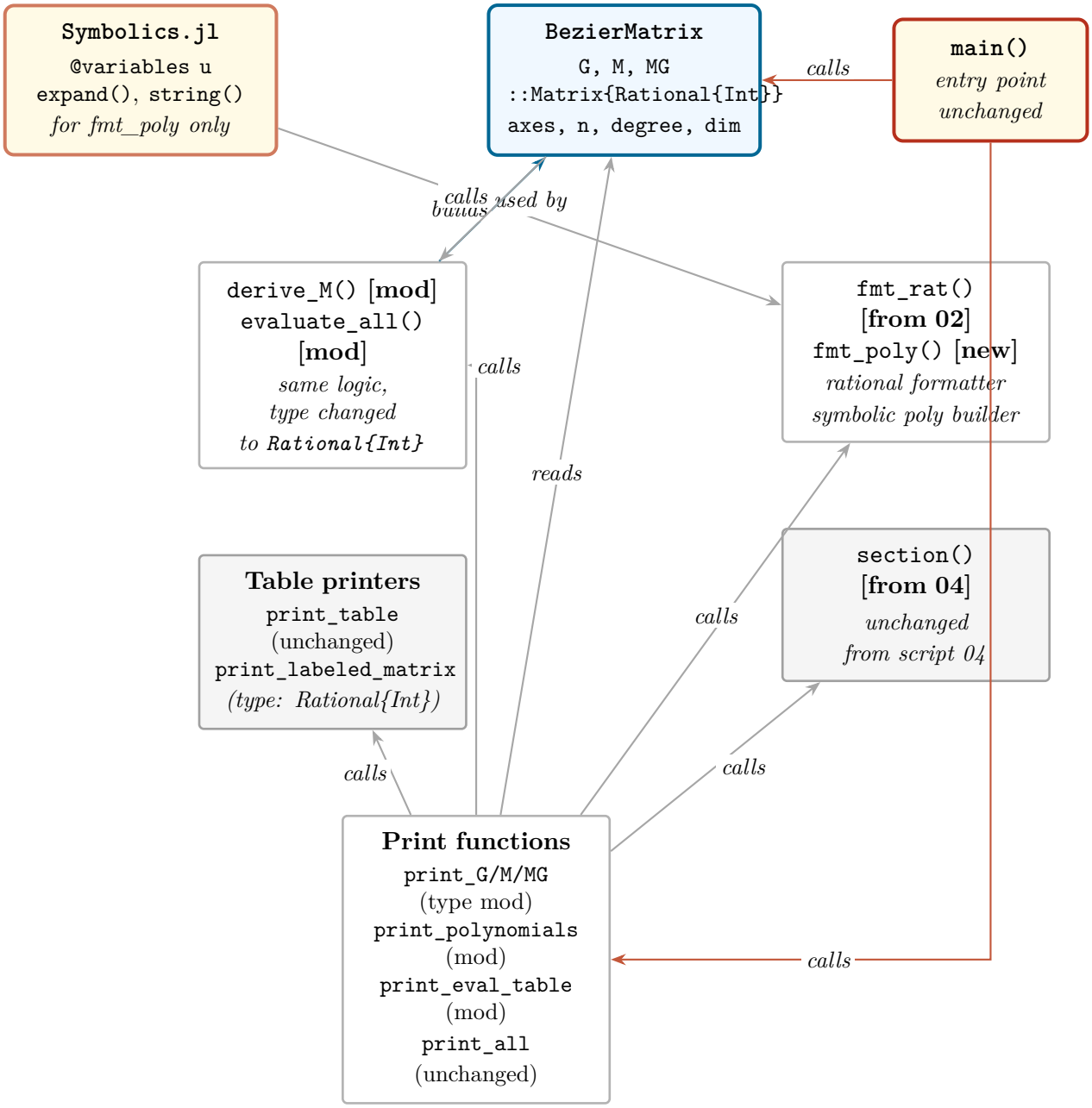
7 `print_labeled_matrix` — Type Change **7**

8 The Print Functions — What Changed **7**
8.1 `print_G`, `print_M`, `print_MG` 7
8.2 `print_polynomials` — uses `fmt_poly` 7
8.3 `print_eval_table` — rational arithmetic path 8

9 Summary: The Hybrid Design **9**

1. Type Map: How the Pieces Fit Together

Script 05 combines the exact arithmetic engine of script 02 with the straight print structure of script 04. In the diagram, grey boxes are inherited unchanged, white boxes are new or modified.



Script 05 is a directed combination of two parents.

From **script 04**: the straight print structure — `section()`, five focused print functions, `print_all`.

From **script 02**: exact rational arithmetic — `Rational{Int}` types, `fmt_rat`,

Symbolics.jl for polynomial display.

The only genuinely new code is `fmt_poly`, which bridges the two parents: it takes a `Rational{Int}` column from `MG` and turns it into a Symbolics.jl symbolic expression.

2. What Is Shared (by Reference)

From script 04 — identical or structurally identical:

Component	Notes
`section()`	Identical.
`print_table`	Identical.
`print_all` signature	Takes `Vector{Rational{Int}}` instead of `Vector{Float64}` — otherwise identical.
`ruler`	Identical.
`main`	Identical logic.

From script 02 — same concept, same name:

Component	Notes
`@variables u`	Identical declaration.
`Rational{Int}` control points	Same typed matrix literal.
`Rational{Int}` u_values	Same exact fraction list.
`fmt_rat`	Same two-method dispatch function (named `fmt_rational` in script 02).

3. The `BezierMatrix` Struct — Type Change

```julia
struct BezierMatrix
    G      ::Matrix{Rational{Int}}
    axes   ::Vector{String}
    n      ::Int
    degree ::Int
    dim    ::Int
    M      ::Matrix{Rational{Int}}
    MG     ::Matrix{Rational{Int}}
end
```

The struct is identical to script 04 except that the three matrix fields changed from `Matrix{Float64}` to `Matrix{Rational{Int}}`.

This single type change propagates everywhere: all arithmetic on these matrices now uses exact fraction arithmetic automatically, because Julia dispatches `+`, `*`, etc. based on the element type.

Why does changing the type of `G` matter?

The constructor computes `MG = M * points`. If `points` is `Matrix{Float64}`, the result is `Matrix{Float64}` — floating-point multiplication. If `points` is `Matrix{Rational{Int}}`, the result is `Matrix{Rational{Int}}` — exact rational multiplication. Julia infers the output type from the input types automatically. No code change in the constructor is needed beyond the type annotation on the struct field.

4. `derive_M` and `evaluate_all` — Type Change Only

4.1. `derive_M`

```
function derive_M(deg::Int)::Matrix{Rational{Int}}
    n = deg
    M = zeros(Rational{Int}, n+1, n+1)
    for i in 0:n
        m = n - i
        for j in 0:m
            k = i + j
            M[n-k+1, i+1] = binomial(n,i) * binomial(m,j) * (-1)^j
        end
    end
    return M
end
```

Two changes from script 04:

1. Return type annotation: `::Matrix{Rational{Int}}` instead of `::Matrix{Float64}`.
2. Initialisation: `zeros(Rational{Int}, n+1, n+1)` instead of `zeros(Float64, n+1, n+1)`.

`zeros(Rational{Int}, n+1, n+1)` creates a matrix filled with `0//1` (the rational zero). The loop body is identical: `binomial(n,i)`, `binomial(m,j)`, and `(-1)^j` all return plain integers, and assigning an integer to a `Rational{Int}` slot auto-promotes it to `binomial_result // 1`.

For $n = 4$, $i = 0$, $j = 0$:

`binomial(4,0) * binomial(4,0) * (-1)^0 = 1 * 1 * 1 = 1`

Assigned to `M[5, 1]` (row `n-k+1 = 5`, col `i+1 = 1`).

Stored as `1//1` in `Matrix{Rational{Int}}`.

Displayed as `1` by `fmt_rat` (denominator is 1).

4.2. evaluate_all

```
function evaluate_all(bm::BezierMatrix,
    u_values::Vector{Rational{Int}})::Matrix{Rational{Int}}
    n    = bm.degree
    Uall = [u_val^(n-k) for u_val in u_values, k in 0:n]
    Uall * bm.MG
end
```

Three type changes from script 04:

- Argument: `u_values::Vector{Rational{Int}}`
- Return annotation: `::Matrix{Rational{Int}}`
- The local variable `u` from script 04 is renamed `u_val` to avoid shadowing the Symbolics.jl symbolic variable `u` declared at the top of the file with `@variables u`.

> **Why rename `u` to `u_val`?**
>
> At the top of the file, `@variables u` creates a symbolic object named `u`. If the loop variable in the comprehension were also named `u`, it would *shadow* the symbolic `u` inside the comprehension body — meaning `u^(n-k)` would use the rational loop value, not the symbolic variable. In this function that would actually be correct, but it would be confusing and fragile. Renaming to `u_val` keeps the two `u`s unambiguously separate throughout the file.

5. fmt_rat — The Rational Formatter

```
fmt_rat(r::Rational{Int}) =
    isone(denominator(r)) ? string(numerator(r)) : string(r)
fmt_rat(n::Integer) = string(n)
```

This is identical in purpose and structure to `fmt_rational` from script 02, renamed for brevity.

The two-method multiple dispatch covers:

- A `Rational{Int}` value: print as `"5"` if denominator is 1, else as `"269//256"`.
- A plain `Integer`: just convert to string.

It is used in three places:

1. `print_labeled_matrix` — to format every cell of G, M, and $M \cdot G$.
2. `print_eval_table` — to format the u value column and the exact result columns.

6. `fmt_poly` — Building a Symbolic Expression

This is the only genuinely new function in the script. It is short but dense — four lines that do something quite different from any formatting function in the previous scripts.

```
function fmt_poly(mg_col::Vector{Rational{Int}}, deg::Int)::String
    expr = sum(
        mg_col[k+1] * u^(deg-k)
        for k in 0:deg
        if mg_col[k+1] != 0
    ; init = 0*u^0)
    string(expand(expr))
end
```

6.1. What it does

Instead of building a string directly from the coefficient vector (as script 04's `fmt_poly` did), this function:

1. Builds a **Symbolics.jl symbolic expression** by summing terms of the form `coefficient * u^power`.
2. Calls `expand()` to canonicalise the expression.
3. Calls `string()` to convert the Symbolics expression to its printed representation.

6.2. The generator sum with `if` filter

```
sum(
    mg_col[k+1] * u^(deg-k)
    for k in 0:deg
    if mg_col[k+1] != 0
; init = 0*u^0)
```

This is `sum` applied to a **generator expression** with an inline `if` filter. It is equivalent to:

```
total = 0*u^0           # initial value
for k in 0:deg
    if mg_col[k+1] != 0
        total += mg_col[k+1] * u^(deg-k)
    end
end
```

The generator computes `mg_col[k+1] * u^(deg-k)` for each `k` where the coefficient is non-zero, and `sum` accumulates them.

> **For the x column: `mg_col = [13//1, -32//1, 24//1, 0//1, 0//1]`, `deg = 4`:**

k	mg_col[k+1]	non-zero?	term built
0	13//1	yes	13//1 * u^4
1	-32//1	yes	-32//1 * u^3
2	24//1	yes	24//1 * u^2
3	0//1	no	skipped
4	0//1	no	skipped

`expr` = symbolic expression for $\frac{13}{1}u^4 + \frac{-32}{1}u^3 + \frac{24}{1}u^2$

After `expand()`: same expression, collected and ordered.

`string(expand(expr))`: `"(24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)"`

Note the term ordering: Symbolics.jl orders from lowest to highest power, which is why u^2 appears before u^4.

6.3. The `init` keyword argument

```
sum(...; init = 0*u^0)
```

`init` provides the **starting value** for the accumulation. Without it, if *all* coefficients are zero (e.g. for a zero polynomial), `sum` would have nothing to add and would throw an error.

`0*u^0` creates the Symbolics expression $0 \cdot u^0$, which simplifies to 0 — a symbolic zero. Using a symbolic zero rather than a plain `0` ensures the result of `sum` is always a Symbolics expression, even when all terms are filtered out.

For the z column: `mg_col = [0//1, 0//1, 0//1, 0//1, 1//1]`:

Only `k=4` passes the filter: `1//1 * u^0 = 1//1 * 1 = 1//1`

`sum` accumulates: `0*u^0 + 1//1 = 1//1`

`expand(1//1)` → symbolic `1`

`string(1)` → `"1"`

Printed as: `z(u) = 1`

Why `0*u^0` and not just `0`?

If `init = 0` (a plain integer) and the first term added is `13//1 * u^4` (a Symbolics expression), Julia would try to compute `0 + (Symbolics expression)`. This works in Julia because Symbolics.jl defines addition with integers, but using `0*u^0` is more explicit: it signals that the accumulator should be a Symbolics expression from the start, avoiding any type ambiguity.

7. print_labeled_matrix — Type Change

```
function print_labeled_matrix(
        mat        ::Matrix{Rational{Int}},
        row_labels ::Vector{String},
        col_labels ::Vector{String})
    rows = [[fmt_rat(mat[r,c]) for c in 1:size(mat,2)]
            for r in 1:size(mat,1)]
    print_table(rows, col_labels; row_labels=row_labels)
end
```

Two changes from script 04:

- `mat` type: `Matrix{Rational{Int}}` instead of `Matrix{Float64}`.
- Formatting function: `fmt_rat` instead of `fmt_fn` (the configurable keyword argument is gone, replaced by the single fixed formatter appropriate for rational values).

Removing the `fmt_fn` keyword argument is possible here because `Rational{Int}` values always use `fmt_rat` — there is no longer a reason to offer an alternative formatter.

8. The Print Functions — What Changed

8.1. print_G, print_M, print_MG

These three functions are structurally identical to script 04. The only difference is that `print_labeled_matrix` now accepts `Matrix{Rational{Int}}`, so the call sites work without any change to the function bodies.

```
function print_G(bm::BezierMatrix)
    section("G  - Geometry Matrix (Control Points)")
    println()
    print_labeled_matrix(bm.G, ["P$(i-1)" for i in 1:bm.n], bm.axes)
end
```

Output: identical to script 04 because all values in G are whole numbers and `fmt_rat` prints `4//1` as `4`.

8.2. print_polynomials — uses fmt_poly

```
function print_polynomials(bm::BezierMatrix)
    section("Resulting Polynomials  (via Symbolics.jl)")
    println()
    for (d, axis) in enumerate(bm.axes)
        println("  $(axis)(u)  =  $(fmt_poly(bm.MG[:,d], bm.degree))")
    end
end
```

The loop body is identical to script 04 but calls `fmt_poly` (the new Symbolics-based formatter) instead of script 04's hand-written `fmt_poly`. `bm.MG[:,d]` passes a `Vector{Rational{Int}}` column to the new `fmt_poly`, which builds the symbolic expression and returns its string representation.

Output:

```
Resulting Polynomials  (via Symbolics.jl)
-----------------------------------------------------------------

x(u)  =  (24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)
y(u)  =  (16//1)*u - (48//1)*(u^2) + (64//1)*(u^3) - (28//1)*(u^4)
z(u)  =  1
```

8.3. print_eval_table — rational arithmetic path

```julia
function print_eval_table(bm::BezierMatrix,
                          u_values::Vector{Rational{Int}})
    section("Evaluation Table  -  P(u) = U * (M * G)")
    pts  = evaluate_all(bm, u_values)
    rows = Vector{Vector{String}}()
    for (r, u_val) in enumerate(u_values)
        row = [fmt_rat(u_val)]
        for c in 1:bm.dim
            exact = pts[r, c]
            dec   = @sprintf("%.4f", Float64(exact))
            push!(row, "$(fmt_rat(exact))  ($dec)")
        end
        push!(rows, row)
    end
    println()
    print_table(rows, vcat(["u"], ["$(ax)(u)" for ax in bm.axes]);
                style=:outline)
end
```

Compare to script 04's equivalent:

Step	Script 04	Script 05
u value display	`fmt(u)` — float to string	`fmt_rat(u_val)` — rational to string
Results from `evaluate_all`	`Matrix{Float64}`	`Matrix{Rational{Int}}`
Exact value per cell	`fmt_frac(v)` — `rationalize()` to recover fraction	`fmt_rat(exact)` — fraction is already exact
Decimal conversion	`@sprintf("%.4f", v)` — float already available	`@sprintf("%.4f", Float64(exact))` — convert rational to float

The key difference is in the exact value column. Script 04 computed with floats and used `rationalize()` to *recover* the fraction. Script 05 never loses the fraction — `exact` is already a `Rational{Int}`, so `fmt_rat` just formats it directly.

Tracing one cell: x at $u = \frac{1}{2}$

`u_val = 1//2` (exact rational, displayed as `"1//2"`)

`pts[3, 1] = 45//16` (from `evaluate_all`, exact rational)

`fmt_rat(45//16)` → `"45//16"`

`Float64(45//16) = 2.8125`

`@sprintf("%.4f", 2.8125)` → `"2.8125"`

Cell string: `"45//16 (2.8125)"`

9. Summary: The Hybrid Design

Script 05 is the shortest in terms of new code — but it demonstrates the most important design principle of the series: **composability**.

The matrix method (scripts 03 and 04) and exact rational arithmetic (script 02) are independent choices. Script 05 shows that combining them requires changing only the type annotations and the one formatting function that bridges the two systems (`fmt_poly`). Everything else — the matrix derivation logic, the evaluation batch, the table printer, the print structure — works unchanged.

Script	Method	Arithmetic
01	Bernstein algebraic	`Float64`
02	Bernstein algebraic	`Rational{Int}` + Symbolics.jl
03	Matrix (pedagogical)	`Float64`
04	Matrix (straight)	`Float64`
05	Matrix (straight)	`Rational{Int}` + Symbolics.jl

Code explanation for `05-bezier-matrix-sympy-text.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated